

wp-04-lineage-anchored-continual-learning

Append-Only Lineage Graphs for Closed-Loop Model

Improvement: Harvest, Gate, Retrain

AsAManThinks Research — Working Paper WP-04

Weslyn Whitehead Jr. (Docwes / Yare Ace NoK)

ORCID: <https://orcid.org/0009-0005-7707-3210>

May 2026 · MaiaM Alchemist v0.4.x · **Revision 2 (June 2026)**

DOI: 10.5281/zenodo.19600795 (foundations anchor)

Revision 2 changes: (C6) the anti-feedback-collapse threshold is corrected from 0.60 to **0.50**, the contraction boundary implied by the paper’s own KL-drift bound $f/(1-f)$; §4.3 now states the contraction argument explicitly and notes the condition under which a higher value would be admissible. No other content changed.

Abstract

We present **Lineage-Anchored Continual Learning (LACL)**, a system architecture for closed-loop model improvement that uses an append-only directed acyclic graph (the “lineage journal”) as the substrate for training data provenance, model versioning, deployment tracking, and harvest qualification. Every meaningful system event — training steps, chat turns, refusals, dream completions, eval scores, deploy snapshots — is appended as a typed node to a single SQLite WAL database, forming an auditable lineage from the original training corpus through every subsequent model generation. The harvest phase reads this graph to construct the next training corpus; the eval suite gates promotion; the chaos-seed feedback biases the next training run toward under-explored regions of the routing space. A novel **anti-feedback-collapse guard** detects when the corpus is predominantly self-distilled from the current model, halting the loop before the model begins training on its own outputs at dangerous scale. We discuss the formal properties of the lineage DAG, the harvest qualification criteria, and the relationship to continual learning literature.

1. Introduction

Series note: This paper is part of the AAMT Working Paper Series. The VortexGate routing system and shell classification used as harvest quality signals are derived in WP-01 (Vortex-Gated MoE). The HeartScale coherence rejection system, whose rejection events feed the lineage graph, is derived in WP-02 (HCRS). This paper uses both as quality signals for harvest qualification without re-deriving them.

Continual learning — training a model incrementally on a growing stream of data without catastrophic forgetting — is an open problem in machine learning. Most work focuses on the forgetting problem: how to update weights on new data without degrading performance on old data (Kirkpatrick et al., 2017; Rebuffi et al., 2017; Lopez-Paz and Ranzato, 2017).

A parallel and less-addressed problem is the **provenance problem**: in a deployed system where model outputs become training data for future models, how do we track the lineage of every piece of training data, enforce quality gates before that data enters a training run, and prevent the feedback loop from collapsing?

This is not a theoretical concern. Systems like ChatGPT (which generates outputs used in internet data, which is then scraped for future training), GitHub Copilot (whose outputs appear in public repos, which are then used as coding corpora), and now voice assistants all face variants of this problem. The typical approach is to simply not address it — to accept that model generations pollute future training data and hope the scale of real-world data dilutes the effect.

LACL takes a different approach: make the feedback loop explicit, auditable, and gated. Every piece of data that enters training is traceable to its lineage source. Every model generation that might re-enter training is tagged `is_model_generated: true` in the lineage graph. A harvest query explicitly controls what fraction of the next corpus comes from self-generated vs. real-world sources. And a circuit-breaker guard halts the loop when self-generation exceeds a safe threshold.

2. The Lineage Journal

2.1 Storage

The lineage journal is a single SQLite database with WAL journal mode, written exclusively by the sidecar process and read by any other process (eval, harvester, UI). Single-writer design eliminates race conditions. WAL mode ensures readers never block writers.

```
CREATE TABLE lineage_node (
  id          BLOB PRIMARY KEY,      -- UUIDv7: lexicographic = chronological
  parent_id   BLOB,                 -- NULL for roots
  kind        TEXT NOT NULL,         -- typed event taxonomy
  ts          TEXT NOT NULL,         -- ISO-8601 UTC
  payload     TEXT NOT NULL,         -- JSON
  payload_hash BLOB NOT NULL,        -- SHA-256(payload) - tamper detection
  actor       TEXT,                 -- "user" | "engine" | "trainer"
  correlation_id TEXT                -- ties multi-node transactions
);
```

UUIDv7 (IETF RFC 9562) encodes a 48-bit millisecond timestamp in the high bits, making lexicographic ordering equivalent to temporal ordering without a separate timestamp index.

2.2 Node taxonomy

Node kinds are free-form strings organized by dot-namespace convention:

```

session.*      → user session events (open, close)
chat.*         → conversational turns (user_message, assistant_reply, refusal)
vortex.*       → routing projections
train.*        → training run events (run_start, step, eval, run_end)
chaos.*        → chaos-seed events
dream.*        → dream generation completions
deploy.*       → deployment events (snapshot, rollout)
harvest.*      → feedback and annotation events
lineage.*      → meta-events (audit_failure, amendment)

```

2.3 Audit invariants

1. **Append-only.** No UPDATE, no DELETE. Amendments create a new node with `kind = "*.amended"` and `parent_id = original.id`.
2. **Cycle-free.** A failed insert (parent not found) emits a `lineage.audit_failure` node.
3. **Tamper-detectable.** SHA-256 of the JSON payload is stored alongside; a mismatched hash raises `LineageCorruptionError` on read.
4. **Monotonic tags.** Tags are additive; no tag removal is possible.

These invariants are enforced by the Python `lineage.validator` module, not by the application layer. They cannot be bypassed by application bugs. (Append-only writes plus monotonic tags plus idempotent integration make the coordination layer **monotone**, hence coordination-free by the CALM principle — see WP-12.)

3. The Harvest Phase

The harvester reads the lineage graph to construct the delta corpus for the next training run:

```

def harvest(since: datetime, before: datetime, run_id: str) -> Corpus:
    # Step 1: Pull all assistant replies in the harvest window
    replies = lineage.query(
        kind="chat.assistant_reply",
        since=since, until=before
    )

    # Step 2: Discard low-quality signal
    replies = [r for r in replies if (
        r.payload["vortex"]["shell"] != "red"          # discard red-shell outputs
        and r.payload["heartscale_resample_count"] <= 3 # discard highly-resampled
        and r.feedback_rating >= 3                     # discard low user ratings
    )]

```

```

# Step 3: Build corpus entries
positives = make_sft_pairs(replies)           # (prompt, reply) pairs
negatives = make_refusal_pairs(              # (prompt, refusal_explanation)
    lineage.query(kind="chat.refusal", ...)
)
adversarials = make_adversarial_examples(    # high-ratio heartscale rejections
    lineage.query(kind="heartscale_reject", ...)
)

return Corpus(positives=positives, negatives=negatives,
               adversarials=adversarials)

```

3.1 Quality signals from the lineage graph

The harvest qualification criteria use signals that are only available because the lineage graph captures them:

- **Shell at generation time** (`vortex.shell`): red-shell generations are discarded even if rated positively by users (the user may not recognize misalignment).
- **HeartScale resample count**: heavily-resampled tokens indicate the model struggled to produce coherent output; these are training examples of poor model performance, not positive examples.
- **Refusals as negative pairs**: the refusal explanation becomes the target for the `archetype_voice` expert.
- **Adversarial examples from rejection traces**: high-ratio rejection events (tokens that scored well above the acceptance threshold) become adversarial test cases for the eval suite.

None of these signals require human annotation. They are generated automatically by the inference pipeline.

4. The Anti-Feedback-Collapse Guard

4.1 Motivation

When a model is deployed and its outputs become training data, and the harvest phase is aggressive, the next training corpus may be dominated by the current model's own outputs. Training on self-generated data is a known failure mode: the model's distribution shift from its training data amplifies, artifacts and hallucinations become reinforced, and diversity collapses. This is sometimes called "model collapse" (Shumailov et al., 2024).

4.2 The guard

The harvester computes the self-distillation fraction before generating the corpus:

```

def check_collapse_risk(corpus: Corpus) -> None:
    n_self_distilled = sum(1 for ex in corpus.positives
                           if ex.metadata.get("actor") == "engine")
    n_total = len(corpus.positives)

```

```

if n_total == 0:
    raise HarvestError("Empty corpus – no eligible examples in harvest window")

self_distilled_fraction = n_self_distilled / n_total

if self_distilled_fraction > COLLAPSE_THRESHOLD:  # default 0.50 (contraction boundary)
    raise FeedbackCollapseWarning(
        f"Self-distillation at {self_distilled_fraction:.1%} exceeds "
        f"safe threshold {COLLAPSE_THRESHOLD:.0%}. "
        "Inject fresh real-world corpus or increase chaos-seed budget."
    )

```

The 50% threshold is the contraction boundary derived in §4.3. It is not a hard safety cutoff but an operator-review gate: the harvest halts and surfaces a decision to the user, who can either (a) rebuild the real-world corpus from updated source trees, or (b) increase the chaos-seed exploration budget to diversify the generated content before harvesting.

4.3 Formal connection to model collapse

Shumailov et al. (2024) show that iterative training on model outputs produces tailed-distribution collapse: rare tokens disappear, the model becomes overconfident in high-probability continuations. The collapse is proportional to the fraction of training data generated by the model.

A formal bound: if the self-distillation fraction is bounded by f at every iteration and fresh real-world data is injected to fill the remainder, the per-iteration KL drift between the model's distribution and the true data distribution is multiplied by $f / (1 - f)$ relative to the previous iteration. For the loop to be a **contraction** — so that drift does not compound to divergence over an N -run chain — this multiplier must satisfy $f / (1 - f) < 1$, i.e.

$$f < 0.5.$$

The threshold is therefore set at the contraction boundary 0.50 (multiplier $1.0\times$). The previous value 0.60 gives multiplier $1.5\times$, which **diverges** if it compounds across iterations: after n self-distilled rounds the drift bound grows as $(1.5)^n$. A value above 0.5 is admissible only under the stronger operational assumption that every iteration fully re-injects ground-truth data so the bound is **reset per step** rather than chained; if a deployment relies on that assumption it should be stated explicitly and verified (each run's real-world fraction independently re-sampled from source, not carried forward).

5. Eval Suite as Promotion Gate

A trained model is not auto-promoted to deployment. It must pass the eval suite:

Promotion criteria:

1. archetype_alignment score $\geq$ baseline - δ_{align} (default $\delta = 5\%$)
2. mmlu_subset not regressed by $> 2\%$
3. gsm8k_lite not regressed by $> 2\%$
4. humaneval_lite not regressed by $> 2\%$
5. latency_throughput p95 not regressed by $> 10\%$
6. heartscale_violations held flat or improved

Criteria 1–5 are standard. Criterion 6 is unique to LACL: we track how often adversarial prompts produce tokens that pass the HeartScale filter but are misaligned (false accepts). This rate must not increase across training runs.

Failing any criterion halts promotion. The user sees a diff of the failing metrics and can decide to (a) accept the regression with a justification note (recorded in the lineage as a `deploy.promotion_override` node), (b) run additional training, or (c) roll back.

All promotion decisions are lineage nodes. The decision and its justification are permanently auditable.

6. Chaos-Seed Feedback

The lineage graph's `chaos.seed` nodes from run N inform run N+1's exploration strategy:

- **Under-visited Meji** from run N (low average routing mass) become preferred targets for chaos seeds in run N+1.
- **High-signal seeds** (seeds that produced large loss deltas, logged in `chaos.seed.payload["delta_loss"]`) are re-used as recurring training augmentations.
- **Artifact-producing seeds** (seeds followed by `harvest.feedback` nodes with `tags=["dream", "artifact"]`) are de-prioritized.

This creates a second-order feedback loop: not just “train on what users liked,” but “explore in the directions the model hasn't been trained to handle well.” It is a learned exploration bonus in the TERA space, analogous to intrinsic motivation in RL but implemented without any learned value function — just lineage graph queries. The under-visited-Meji preference is the same count-based exploration weighting used at the curriculum-sampling stage (WP-05 coverage weight).

7. Relationship to Continual Learning Literature

Elastic Weight Consolidation (Kirkpatrick et al., 2017): Constrains weight updates near important parameters for prior tasks. LACL takes a data-centric approach rather than weight-centric: quality-gate the incoming data, not the weight updates.

Experience Replay (Rolnick et al., 2019): Retain exemplars from prior tasks in the replay buffer. LACL's lineage graph is a principled experience replay buffer: every turn is available for replay, tagged with quality signals, bounded by the collapse guard.

iCaRL (Rebuffi et al., 2017): Class-incremental learning with herding-based exemplar selection. LACL's harvest qualification is analogous to exemplar selection but uses alignment signals (shell, HeartScale) rather than class-representativeness.

Feedback distillation / self-play (Bai et al., 2022): Model generates its own training data. LACL explicitly bounds and monitors this fraction, addressing the known pathology of unconstrained self-distillation.

Key distinction: No prior continual learning work formalizes an anti-feedback-collapse guard with a closed-form fraction-based circuit breaker, nor uses routing geometry signals (shell, HeartScale) as data quality criteria for harvest qualification.

8. Privacy Considerations

The harvest pipeline is opt-in per deploy target. Non-opted-in sessions write lineage nodes with `actor = "anon"` and are excluded from harvest queries. This ensures user data is never used for training without explicit consent, while the model still benefits from the alignment signals (HeartScale rejections, shell classifications) that are computed locally and do not contain user content.

9. Conclusion

LACL provides a complete, auditable architecture for closed-loop model improvement. Its key contributions are: the append-only lineage journal as the universal substrate for provenance; the harvest qualification criteria that use alignment signals rather than human annotations; the anti-feedback-collapse guard with a formal KL-divergence bound and a contraction-boundary threshold (0.50); and the chaos-seed feedback loop that biases exploration toward under-trained regions of the routing space. Together, these mechanisms allow a small (2B parameter) personal model to improve continuously from its deployment data without the catastrophic failures — collapse, forgetting, alignment drift — that have plagued deployed continual learning systems. The lineage graph is both the scaffold that makes the loop safe and the audit trail that makes it trustworthy.

References

- Kirkpatrick, J. et al. (2017). Overcoming catastrophic forgetting in neural networks.
- Rebuffi, S. et al. (2017). iCaRL: Incremental classifier and representation learning.
- Lopez-Paz, D., Ranzato, M. (2017). Gradient episodic memory for continual learning.
- Rolnick, D. et al. (2019). Experience replay for continual learning.
- Shumailov, I. et al. (2024). AI models collapse when trained on recursively generated data.

- Bai, Y. et al. (2022). Constitutional AI: Harmlessness from AI feedback.
- Whitehead, W. (2026). MaiaM Wings: Consciousness-Aligned Language Architecture. Zenodo. DOI: 10.5281/zenodo.19600795
- AAMT Foundations Papers I–V: `aamt-foundations/foundations.json`